

Functional Coverage for OpenQASM 3.0

The bug that started it, and the
power of knowledge

Jeff Heckey (AWS Braket)
2026-07

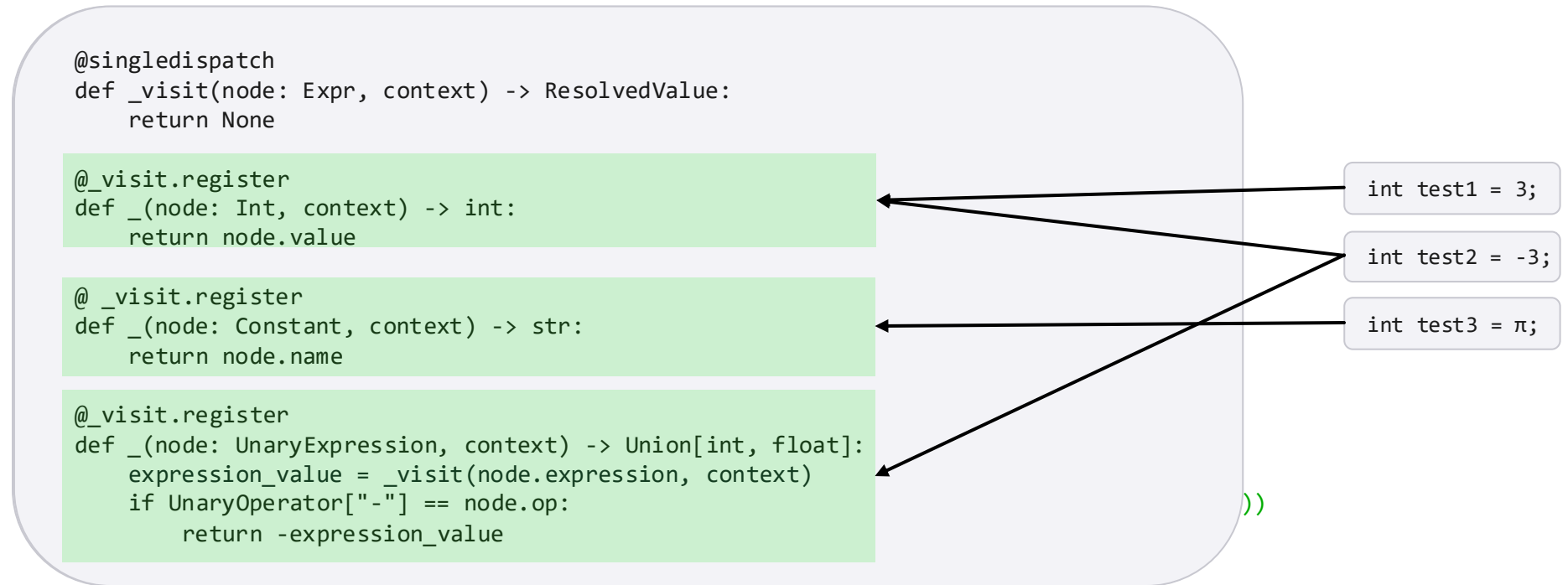


The bug: $\text{rx}(-\pi)$

- A customer used $\text{rx}(-\pi)$ in a circuit – it failed and paged our on-call
- **Bug:** AST walker returned “ π ”, not $3.14159\dots$ → we tried to negate a string
- **Root Cause:** we had no unit test for negating a built-in constant
- **Further Challenge:** we value customer’s privacy, so we can’t simply look at your circuits to determine the root cause. Any complex bug may not be easily reproducible.
- **The Need:** How do we know that every possible test case is written?

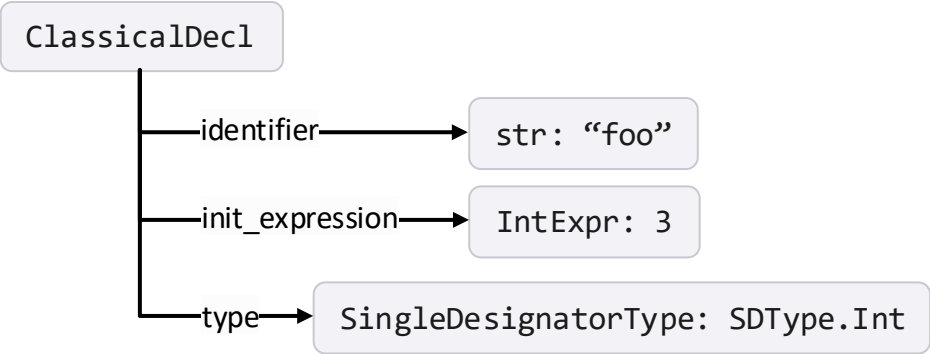
Code coverage is not enough

- Line/branch code coverage measures the **compiler's source**, not the **language**.
- You can hit **100% code coverage** and still crash on valid input



AST → rows: one node, one row

Example: `int x = 3;`



nodes table (real rows from `coverage.db`):

id	type	parent	field
37535	ClassicalDecl	37534	statements
37536	IntExpr	37535	init_expression
37537	SingleDesignatorType	37535	type

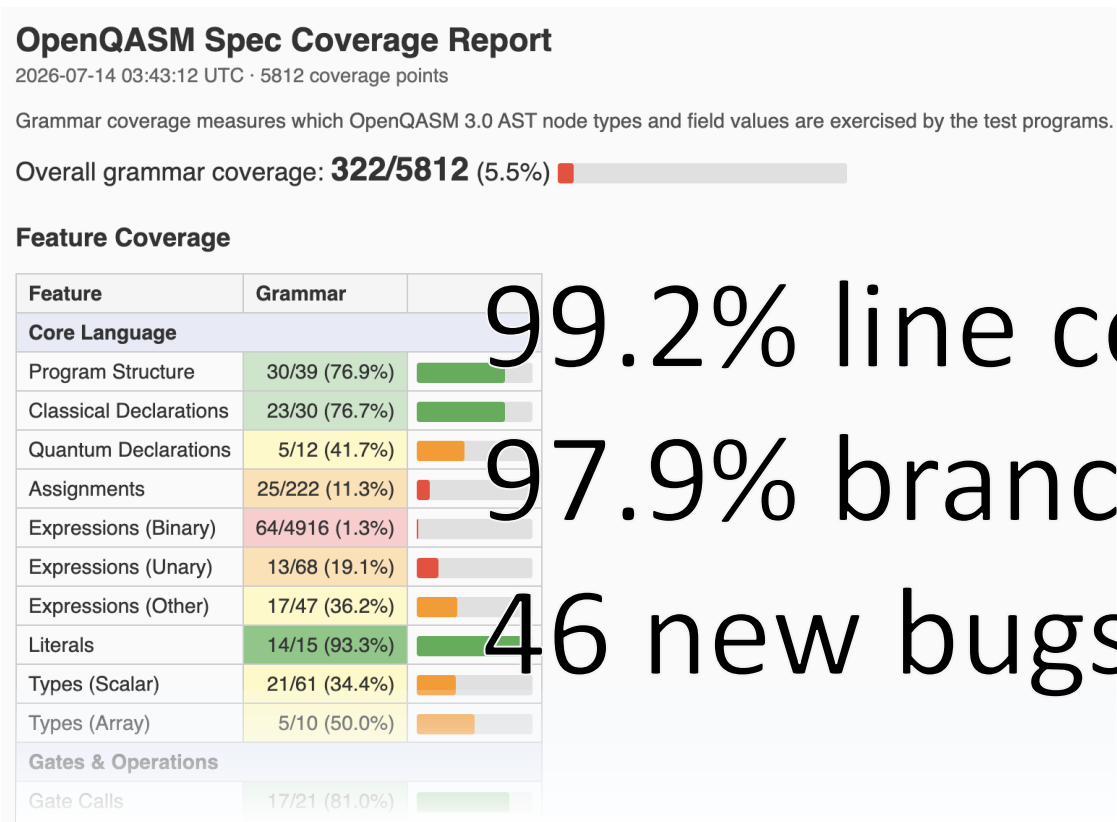
The **shape**, **type**, and **value** are used as coverage points

node	field name	shape	value	→ coverage point
ClassicalDecl	init_expression	optional	present	ClassicalDecl.init_expression.optional.present ✓
IntExpr	value	scalar	3	IntExpr.value.* ✓
SingleDesignatorType	type	scalar	SDType.Int	SingleDesignatorType.type.SDType.Int✓
SingleDesignatorType	size	optional	absent	SingleDesignatorType.size.optional.absent ✓

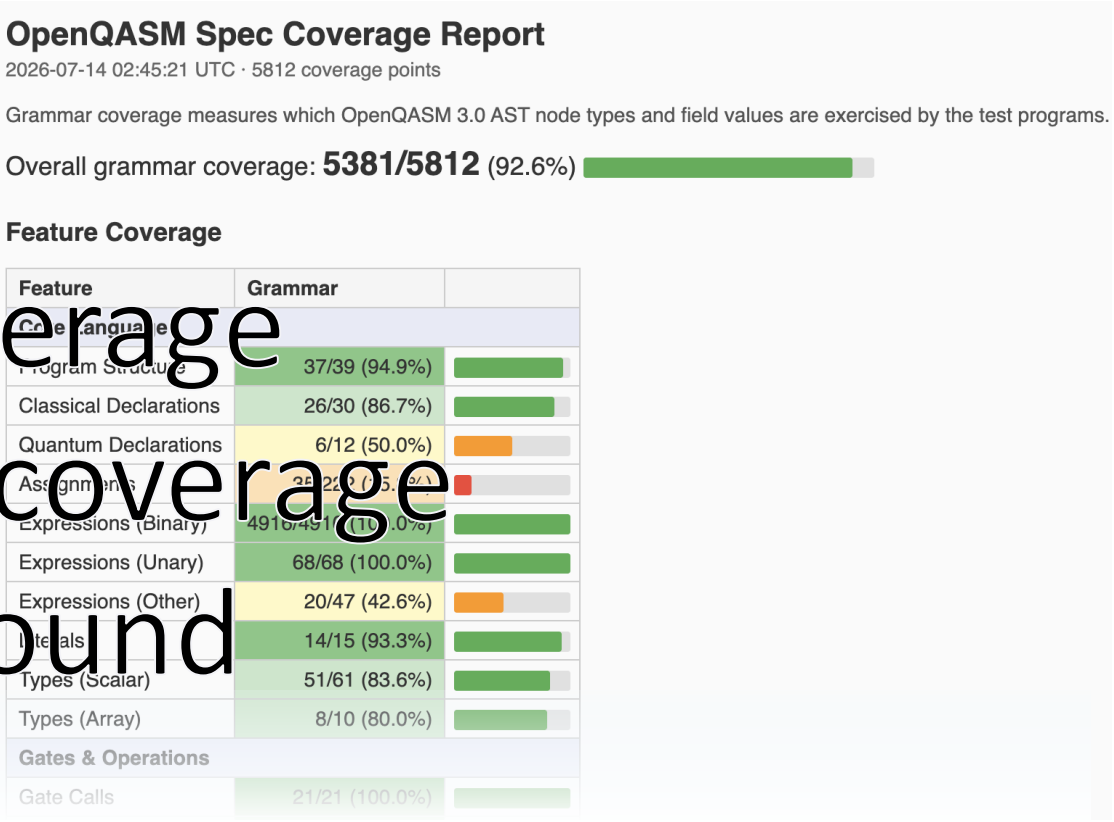


And now we can know...

Before



After



99.2% line coverage
97.9% branch coverage
46 new bugs found



Where's it going?

- **Gap-filling** the remaining uncovered points is now a closed-form question
- Since the **uncovered points** are known and clearly defined, generating unique test cases can easily be done with AI or fuzz testers
- The **cross-coverage syntax** enables subtree descent for more complex semantic coverage points (identifying recursive functions, bounds checking for min/max integers and overflows, etc.)
- The tool is **language-agnostic**: any **EBNF grammar** or a **static AST definition** can be used to generate the structural framework for the coverage points



Thank you!

